GM ECU Simulator - User Manual

User Manual

A guided start for new users

Edition: 2026-06

Contents

About this manual

This manual is written for a brand-new user. You do not need to know what J2534 is, what DPS is, or how an ECU is programmed in the real world. By the end of the first three sections you will have the software installed, registered, and producing traffic against a real tester application on your PC.

How to read this manual:

- **New users:** read Sections 1 to 4 in order, then pick the Quickstart for the mode you actually want to use (Section 5, 6, or 7).
- **Returning users:** the Reference (Section 11) and Troubleshooting (Section 10) sections are the day-to-day pages.
- **Looking for something specific:** the Glossary (Section 12) explains every acronym used in the app and in this document.

> **A note on the two big concepts**
>
> Section 2 introduces two ideas you will see everywhere in the app: the mode (what is being simulated - ECU Simulator or DPS Simulator) and the connection (how a host reaches it - J2534 or raw-CAN TCP). Read that section before clicking around - those two choices, made together in one dropdown, are the single biggest source of confusion for new users.

1. What this software does

GM ECU Simulator is a Windows application that pretends to be one or more GM vehicle ECUs (the small computers that run an engine, transmission, or other vehicle subsystem). It speaks the same diagnostic protocols a real ECU speaks, over the same plumbing a real diagnostic tool uses, but no vehicle and no physical interface hardware is involved.

In practical terms it lets you:

- Test a diagnostic application against a virtual ECU on the same PC, with no Tactrix, MongoosePro, or any other hardware plugged in.
- Reproduce a programming session end-to-end against the simulator. GM's internal Diagnostic Programming System (DPS) and the popular open-source flash tools both run against this simulator and complete with a success message.
- Capture every byte that a tool sends during a session. The simulator writes a .bin of the data downloaded into each programming session and a CSV log of every CAN frame in both directions, with human-readable annotations.

- Develop and debug your own diagnostic application. Point it at the simulator instead of a real car, iterate quickly, and avoid the wait-burn-replace loop with a real ECU.

How it pretends to be real hardware

Diagnostic applications on Windows connect to vehicle hardware through a standard interface called J2534. A J2534 device (the Tactrix OpenPort, MongoosePro, Bosch MDI, and dozens of others) is essentially a USB-to-CAN dongle with a small native DLL on the PC side that the diagnostic application loads. The application calls 14 functions in that DLL (PassThruOpen, PassThruWriteMsgs, PassThruReadMsgs, etc.) and the dongle does the rest.

GM ECU Simulator ships its own small native DLL with the same 14 functions. When you register the simulator (a one-click step you do once), Windows tells every J2534-aware application on the machine that there is a new device available called GM ECU Simulator. The application loads the simulator's DLL just like it would load a Tactrix DLL, and from then on every read or write the application makes goes to the running simulator instead of to a real vehicle.

> **[SCREENSHOT PLACEHOLDER]**
>
> **File:** docs/manual/screenshots/01-main-window-overview.png
>
> **Caption:** The main window after installation, with no ECUs configured.

2. Modes and connections

One dropdown in the top-right of the main window controls two things at once: the mode (what the simulator is pretending to be) and the connection (how a host program reaches it). The dropdown lists three picks: "ECU Simulator - J2534", "ECU Simulator - TCP", and "DPS Simulator".

> **[SCREENSHOT PLACEHOLDER]**
>
> **File:** docs/manual/screenshots/02-mode-selector.png
>
> **Caption:** The mode dropdown in the top-right corner.

The two modes

ECU Simulator. The sandbox. You invent one or more virtual ECUs from scratch, each at its own CAN ID, and define the PIDs they report. Values come from the signal layer - a live engine model driven by a scenario (idle, cruise, an accel-decel sweep) and an engine character (naturally aspirated or boosted V8) - or, per PID, from a plain waveform or a file replay. State persists to a JSON file in the app's local data folder, so your ECUs come back next launch. This is also the mode you use to emulate a target for an open-source flash tool (see Section 7).

DPS Simulator. A single-ECU programming workflow. You point the Prime-From-Archive wizard at a real GM DPS programming archive (.zip); the simulator parses the utility-file bytecode and the calibration files inside, scans the OS module to extract the PID table, decides which security-access algorithm the archive expects, and builds a virtual ECU that satisfies all of it. You then launch DPS, pick GM ECU Simulator from its J2534 device list, and run the session through to "Programming Successfully Ended". (This single mode folds together what older versions split into separate DPS Write and DPS Read modes.)

> **DPS is GM's internal programming tool**
>
> DPS (Diagnostic Programming System) is the dealership-side application GM uses to reflash vehicle modules. A DPS archive is the bundle that defines what a single programming job does: which kernel to upload, which calibration files to flash, which validation routines to run. The archive format is reverse-engineered for the simulator's purposes - you do not need a DPS license to use this mode.

The two connections

"Connection" is how an outside program reaches the virtual bus. It is a separate idea from the mode - the same ECU Simulator setup can be reached either way:

- **J2534 (the default).** The simulator registers itself as a standard J2534 PassThru device. Real diagnostic hosts - Tech 2 Win, GDS, DPS, your own scan tool - load its native shim DLL and talk to it exactly as they would a Tactrix or MongoosePro. This is the connection you want for anything that already speaks J2534.
- **TCP (raw CAN).** Instead of J2534, the simulator opens a localhost TCP socket that carries raw CAN frames. A program that is not a J2534 host - for example a separately developed gauge or data-logger simulator - can connect to that socket and join the same virtual bus as if it were another node on the wire. ISO-TP runs on both ends; the socket only ever carries single CAN frames. DPS Simulator is J2534-only (a raw-CAN

gauge has no role in a programming session), which is why it has no "- TCP" variant in the dropdown.

Which one should I pick?

I want to...	Pick this dropdown entry
Develop my own J2534 scan tool / data logger and read PIDs from a virtual ECU	ECU Simulator - J2534
Run an open-source flash tool (DPS aside) against a virtual ECU	ECU Simulator - J2534
Connect a non-J2534 program (e.g. a gauge sim) to the bus over a socket	ECU Simulator - TCP
Have a real DPS programming session complete successfully on my desk	DPS Simulator

3. System requirements and installation

Prerequisites

- **Operating system:** Windows 10 or Windows 11, 64-bit.
- **.NET 9 Desktop Runtime:** free download from Microsoft. The app's installer will prompt you if it is missing.
- **Visual C++ 2015-2022 Redistributable:** both 32-bit (x86) and 64-bit (x64) versions, because the simulator ships a 32-bit and a 64-bit native shim.
- **Administrator rights for one step:** registering the simulator as a J2534 device writes to the Windows registry under HKLM\SOFTWARE\PassThruSupport.04.04. The app will prompt you (UAC) at exactly the right moment - you do not need to launch the app as Administrator.

Step-by-step installation

1. Unzip the release archive into a folder of your choosing. A typical location is C:\Program Files\GM ECU Simulator, but any folder works - the app does not assume a specific install path.
2. Double-click GmEcuSimulator.exe to launch the app. The main window opens and the J2534 status pill in the top-right will read "Not registered".
3. Open the J2534 menu and click **Register as J2534 device...** Windows will display a UAC prompt because registering writes to HKLM. Click Yes.
4. After the UAC dialog closes the status pill updates within a second or two. You want to see "Registered (32-bit + 64-bit)".
5. Sanity check: open the J2534 menu again and click **Show registered devices...** You should see "GM ECU Simulator" listed under both the 64-bit and 32-bit registry views, each pointing at the matching shim DLL in your install folder.

[SCREENSHOT PLACEHOLDER]

File: docs/manual/screenshots/03-j2534-menu.png

Caption: The J2534 menu open, showing the four registration-related items.

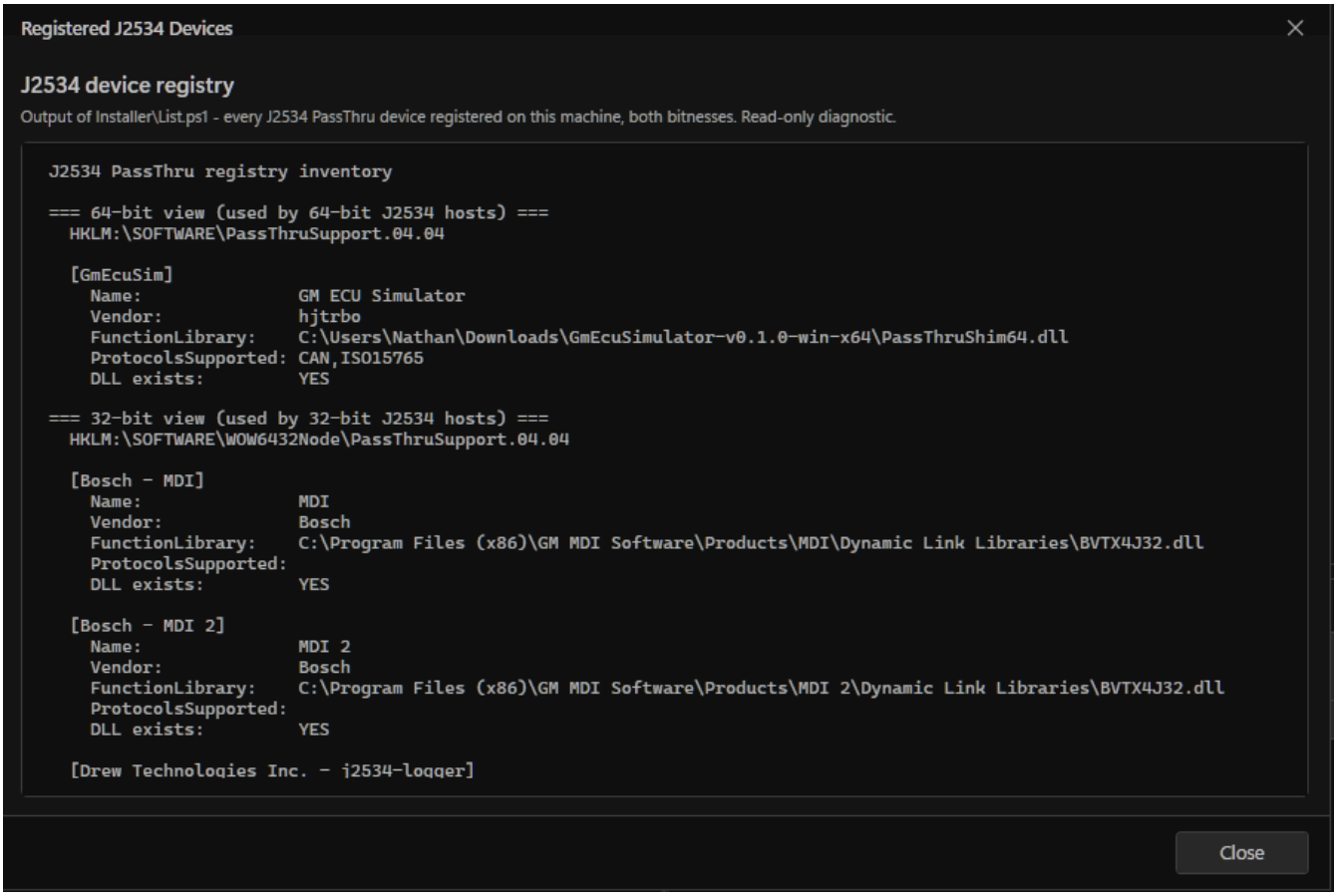

Figure: The Registered Devices window after a successful registration. The simulator appears under both 64-bit and 32-bit views.

> **Why two bitnesses?**
>
> Windows cannot mix bitnesses in a single process: a 32-bit application can only load a 32-bit DLL, and a 64-bit application can only load a 64-bit DLL. Most older diagnostic apps (Tech 2 Win, GDS, Bosch MDI, Tactrix OpenPort, DPS itself) are 32-bit. Newer tools (and the sibling GM DataLogger project) are 64-bit. Registering both bitnesses means the simulator is available to every J2534 host on your PC regardless of the host's architecture.

Removing the registration

Open the J2534 menu and click **Unregister**. This removes only the GM ECU Simulator entries from the registry; no other vendor's J2534 device is touched. The app itself stays installed; you can re-register at any time by clicking Register again.

4. Tour of the main window

The whole app lives in one window. Here is a tour from top to bottom.

> **[SCREENSHOT PLACEHOLDER]**
>
> **File:** docs/manual/screenshots/05-main-window-annotated.png
>
> **Caption:** The main window with the major regions labelled. (Suggested annotation overlay: 1 Titlebar, 2 Status pills, 3 Menu bar, 4 Mode selector, 5 Live PID tile dashboard, 6 Workspace tabs, 7 Status bar. ECU and PID editing is in the separate ECU Editor window, not shown here.)

Titlebar and status pills

Two pills sit on the right side of the titlebar:

- **Selected-ECU security pill (left):** shows the $27 SecurityAccess state of the currently-selected ECU - green when unlocked, amber when locked, red when in a failed-attempt lockout. Only appears when an ECU is selected.
- **J2534 registration pill (right):** shows whether the simulator is currently registered with Windows as a J2534 device. Green dot = registered, amber/red = not registered or partially registered.

Menu bar

Six top-level menus. The most commonly-used items:

Menu	Notable items
File	New, Open, Save, Save As, Clear primed archive, Exit. Ctrl+N / Ctrl+O / Ctrl+S work as you would expect.
J2534	Register, Unregister, Show registered devices, Reset IPC pipe (use if a host disconnects messily and the simulator stops accepting new connections).
ECU	Open ECU settings, Reset ECU State (power-cycle every ECU on the bus), Drive HW $3E keepalives (toggle whether the simulator runs the tester-present timer on the host's behalf).
Tools	Developer utilities, including Add blank ECU.
Log	Master gate for file logging plus four toggles that decide what reaches the log file (J2534 calls, bus traffic, description tags, collapsed bulk transfers).
Theme	Pick a colour palette (see Section 9).

All per-ECU editing happens in the modeless **ECU Editor** - open it from **ECU > Open ECU settings...** or with Ctrl+Shift+P (see "The ECU Editor" below).

Mode selector

Top-right of the menu bar. The dropdown has three entries - "ECU Simulator - J2534", "ECU Simulator - TCP", and "DPS Simulator" - each pairing a mode with a connection (Section 2). Switching mode wipes the current ECU list and prompts to save first if you have unsaved changes; switching only the connection (J2534 <-> TCP) keeps your ECUs and just swaps the transport underneath them.

Live PID tile dashboard

The body of the main window is a read-only dashboard of live PID tiles for the selected ECU - the PIDs you have pinned, each showing its current synthesised value refreshing about ten times per second. It is for watching values at a glance; all editing happens in the ECU Editor. (In DPS Simulator mode the dashboard collapses, since a programming target is not something you watch PID-by-PID.)

> **[SCREENSHOT PLACEHOLDER]**
>
> **File:** docs/manual/screenshots/06-live-tile-dashboard.png
>
> **Caption:** The main-window live PID tile dashboard for the selected ECU.

The ECU Editor

Everything you configure about an ECU lives in this separate, modeless window (open via **ECU > Open ECU settings...** or Ctrl+Shift+P). It has its own ECU list with a toolbar - Add, Remove, Save ECU (to a .ecu.json), Load ECU - so you manage the whole bus from here; there is no ECU list on the main window itself. Selecting an ECU in the editor's list does not disturb which ECU the main-window dashboard is showing.

For the selected ECU the editor exposes:

- **ECU settings:** Name, the CAN IDs (request / USDT response / UUDT response), the engine character and scenario, and - for a DPS-primed ECU - the "Selected ECU" header and the "Edit prime..." button.
- **PID table:** one row per PID, with its Mode ($22 / $1A / $2D), Address, Name, the Signal column (value source), scaling, unit, and size. The Live column shows the current value.
- **Waveform panel:** the generator for the selected PID (sin / triangle / square / sawtooth / constant / file replay), used when that row's value source is Waveform.
- **Advanced:** per-ECU diagnostic-stack settings - flow control (FC.BS / FC.STmin), the $27 security module, and "Load identity from bin..." to pull real identity DIDs out of a flash readback.

Adding an ECU is mode-aware. In ECU Simulator mode, Add inserts a blank ECU at the next free OBD-II ID ($7E0/$7E8 for the first, $7E1/$7E9 for the second, and so on) - have as many as you like. In DPS Simulator mode, Add opens the Prime-From-Archive wizard (Section 6) and only one primed ECU is allowed at a time.

[SCREENSHOT PLACEHOLDER]

File: docs/manual/screenshots/07-ecu-editor.png

Caption: The ECU Editor: ECU list, settings, PID table, and waveform panel.

Workspace tabs

The bottom half of the window is a tab control. The tabs that are visible depend on the current mode:

- **Bus log:** always visible. Two side-by-side text panes - CAN frames (left) and J2534 calls (right). Use the toolbar checkbox "Log traffic" to start/stop the live capture into the panes; "Hide $3E" suppresses tester-present keepalives so the log is readable.
- **Bin Replay:** visible in ECU Simulator mode only. Load a .bin produced by the sibling GM DataLogger to replay recorded channel data through the ECU's PIDs.
- **Glitch settings:** visible in ECU Simulator mode only. UI for per-service probability of injected NRCs / dropped frames / corrupted bytes. (Note: defined but not yet wired into the bus logic.)
- **Captures:** visible in DPS Simulator mode. Browser for the .bin captures the simulator wrote automatically during programming sessions.

Status bar

Bottom of the window. Shows the current status text on the left ("Idle", "Listening", "Connected", error messages), and on the right the primed archive name (DPS Simulator mode) plus, on the J2534 connection, the named-pipe path the shim DLL forwards to: \\.\pipe\GmEcuSim.PassThru.

5. Quickstart - ECU Simulator

Goal of this section: get a virtual ECU on the bus, configure one PID, and read its value from a J2534 host application.

Step 1 - Pick ECU Simulator mode

1. In the top-right of the menu bar, set the dropdown to "ECU Simulator - J2534". If you are already in another mode the app prompts to save the current configuration first - say Yes or No depending on whether you want to keep it. (Pick "ECU Simulator - TCP" instead if your host connects over the raw-CAN socket rather than J2534 - everything else in this quickstart is identical.)

Step 2 - Open the ECU Editor and add a virtual ECU

1. Open the ECU Editor: ECU > Open ECU settings... (or press Ctrl+Shift+P). This modeless window is where all ECU and PID editing happens.
2. Click + in the editor's ECU toolbar. A new ECU appears in the list (default name "ECU1", request CAN ID $7E0, USDT response $7E8, UUDT response $5E8). Click it once to select it.
3. In the ECU settings, rename it (the Name field) and adjust the CAN IDs if you need to match a specific tester's expectations. Most testers default to $7E0/$7E8 and $5E8, so the default is usually correct.

Step 3 - Add a PID

1. In the editor's PID table, click + to add a row. Leave Mode as $22, set Address to $1000 (or any 16-bit value), Name to "My Test PID", Size to Word, and a scaling formula such as raw * 0.1 to convert the raw 16-bit value to a real-world unit.
2. Pick the row's **value source** in the Signal column. You have three choices: attach one of the engine model's named signals (Engine RPM, MAP, Coolant Temp, ...) so the value tracks the running scenario; choose Waveform to drive it from the row's own generator; or leave it None for a flat zero. For this walkthrough, choose the Waveform option.
3. In the Waveform panel (right side of the editor), pick Sin, set Amplitude to 5000, Frequency to 0.5 Hz, and Offset to 5000. The PID will report values that sweep between 0 and 10000 raw counts, or 0 to 1000 once scaled.
4. The Live column in the editor - and the live PID tile dashboard back on the main window - now shows the PID's current sampled value, refreshing about 10 times a second.

[SCREENSHOT PLACEHOLDER]

File: docs/manual/screenshots/07-ecu-editor.png

Caption: The ECU Editor with one PID configured and a sine waveform attached.

Make the whole ECU behave like a running engine

Instead of (or alongside) per-PID waveforms, set the ECU's engine character (Naturally Aspirated V8 / Boosted V8) and scenario (Key-On Engine-Off / Idle / Cruise / Accel-Decel Sweep) in the ECU Editor's settings. Every PID whose value source is a Signal then moves together and stays mutually consistent - RPM, MAP, airflow, spark, and fuel trims all reflect the same operating point - the way they would on a real engine.

Step 4 - Connect a J2534 host

1. Launch any J2534-aware application on your PC. For example, the sibling GM DataLogger project, or any third-party scan tool.
2. Open its device-selection dialog and pick "GM ECU Simulator". The simulator's status bar should update to show a connected client.
3. Have the host read PID $1000 from the ECU at CAN ID $7E0. The Bus log tab will show the request frame coming in and the response frame going out, both annotated.

> **Tip - turn on file logging early**
>
> The Log menu's "Log to file" option writes every frame to a CSV in %LOCALAPPDATA%\GmEcuSimulator\logs\bus logs\. Turn it on before you start any serious testing - it is much easier to grep through the CSV than to scroll the live textbox after the fact.

6. Quickstart - DPS Simulator

Goal of this section: prime a virtual ECU from a real DPS archive, run a real DPS programming session against it, and see the captured downloaded data.

Step 1 - What you need

- A DPS programming archive: a .zip file containing one utility file (the bytecode that drives the session) and one or more calibration files. The simulator has been validated against two such archives as of the current release.
- A working installation of GM DPS on the same PC, or another DPS-compatible programming tool. The simulator only emulates the ECU side; you still need a tool to drive the session.

Step 2 - Pick DPS Simulator mode

1. In the top-right of the menu bar, set the dropdown to "DPS Simulator". The window reconfigures: the live-tile dashboard collapses and the Captures tab appears at the bottom. (DPS Simulator is J2534-only, so there is no connection suffix to choose.)

Step 3 - Run the Prime-From-Archive wizard

1. Open the ECU Editor (ECU > Open ECU settings..., or Ctrl+Shift+P) and click + in its ECU toolbar. In DPS Simulator mode this opens the DPS Prime Wizard at Page 1: Archive.

2. Click **Browse for archive...** and pick the .zip. The wizard parses it on the spot and shows the archive name, utility file name, calibration file count, and (if present) the OS part number extracted from the OS module header.

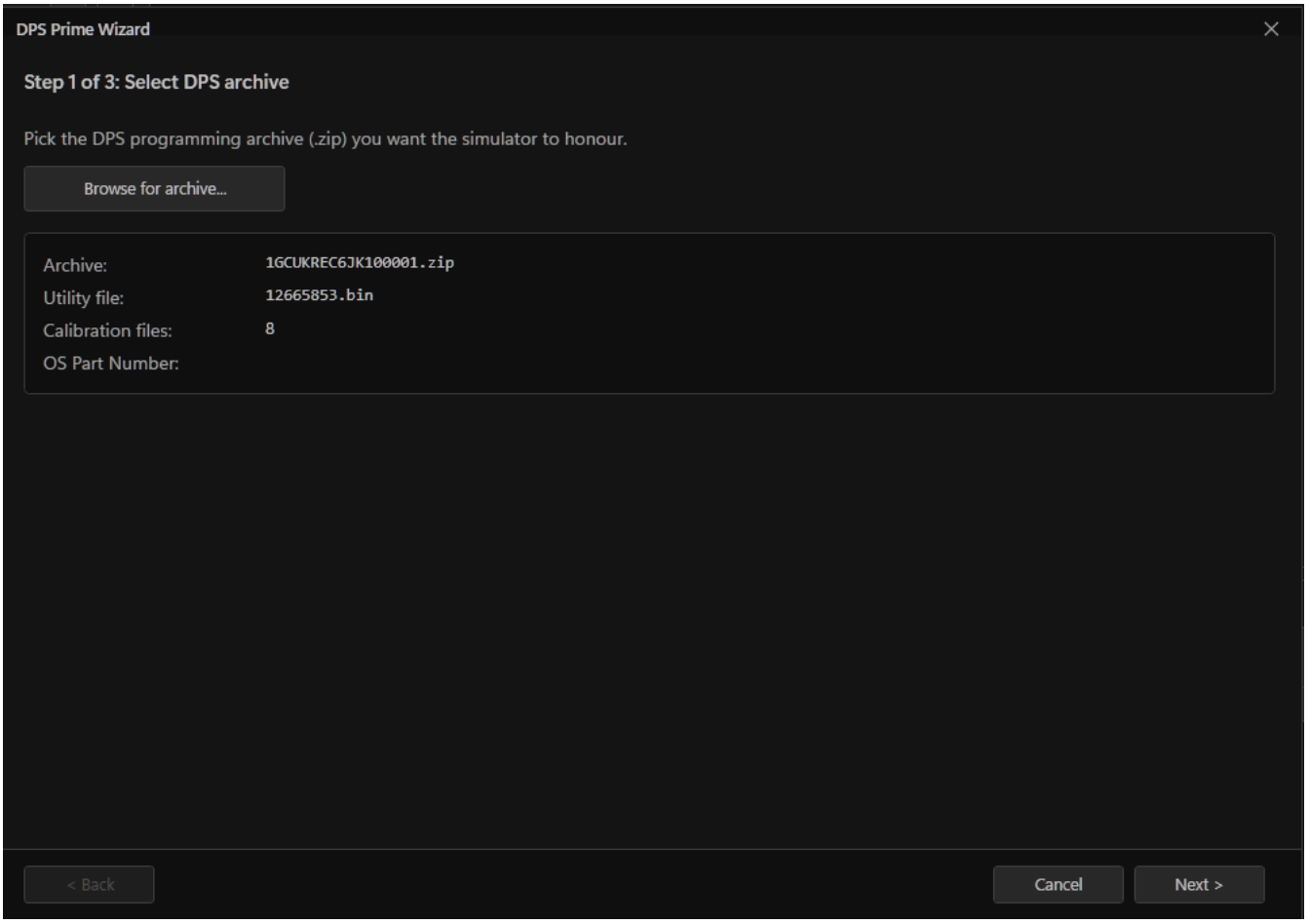

Figure: Prime Wizard - Page 1: archive loaded successfully.

1. Click Next to advance to Page 2: Phase 3 Review. This page shows the post-flash configuration steps the wizard inferred from the utility-file bytecode - which DIDs the tester will write, which PIDs it will read back, which security-access algorithm it will use, and so on.

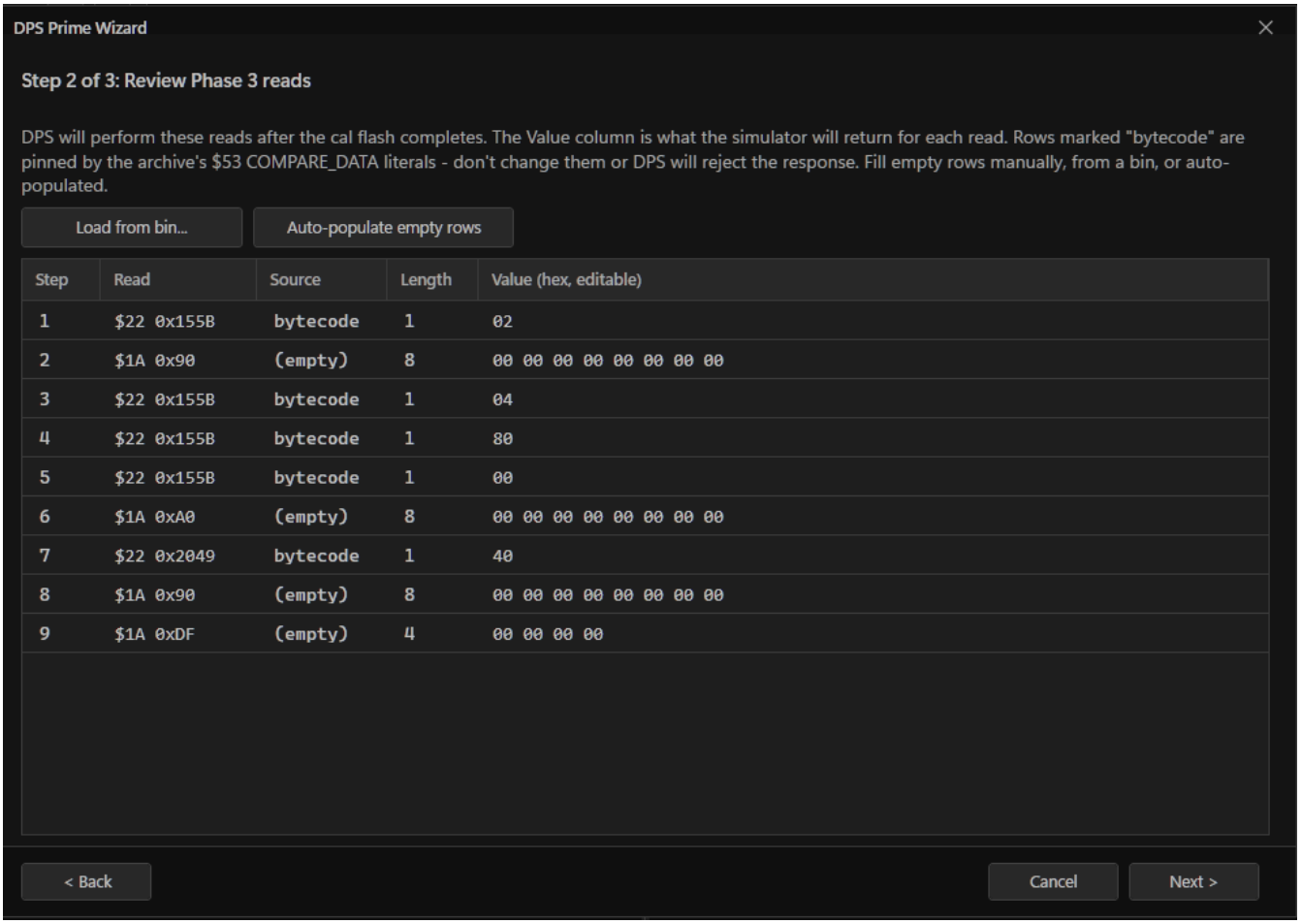

Figure: Prime Wizard - Page 2: Phase 3 review.

1. Click Next to advance to Page 3: Commit Summary. This page is the final review before priming. Confirm everything looks right.

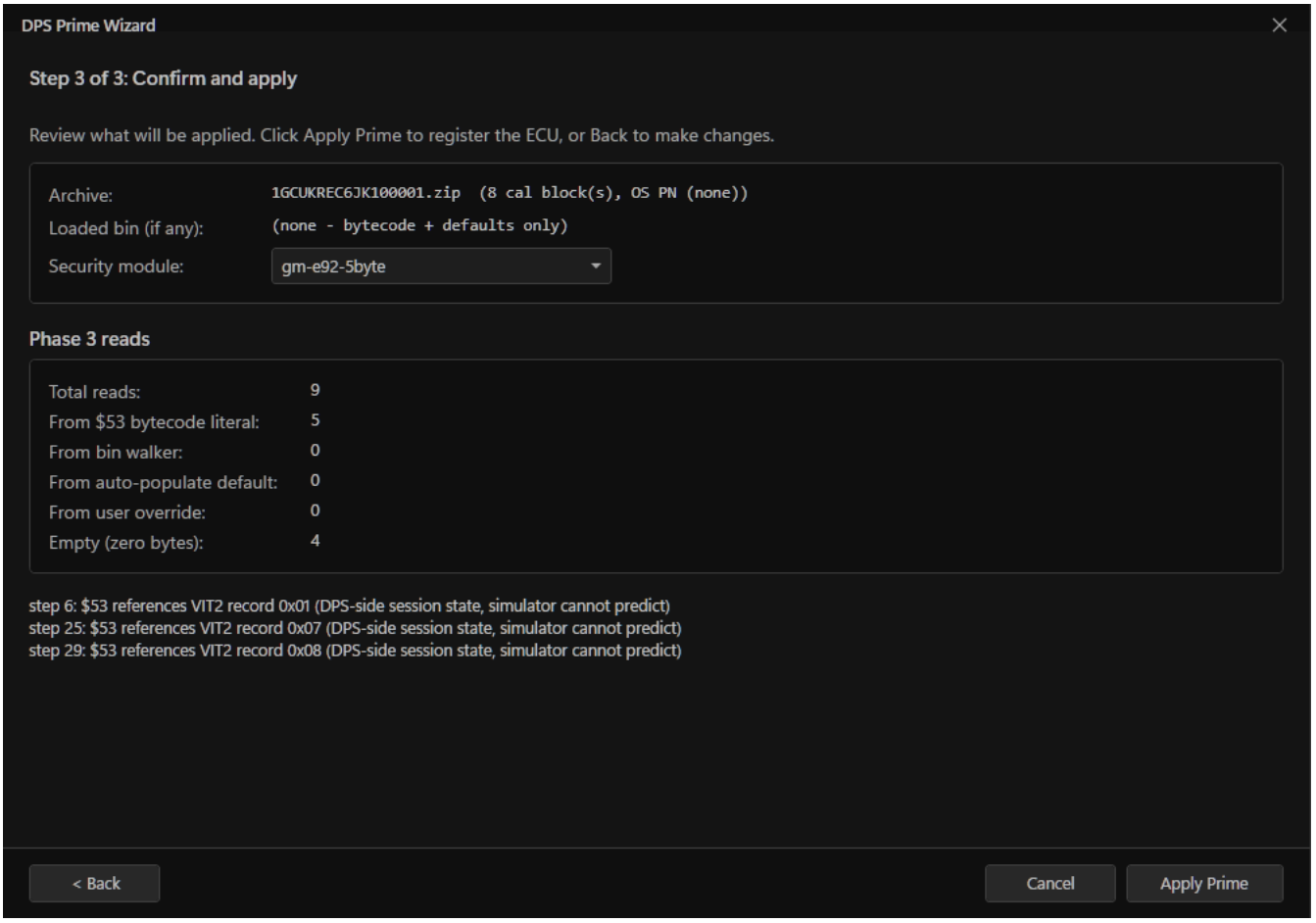

Figure: Prime Wizard - Page 3: commit summary.

1. Click **Apply**. The simulator builds the virtual ECU and registers it at the standard OBD-II CAN IDs. The wizard closes; the new ECU appears in the ECU Editor's list with its configuration filled in. A full report of the prime is written to %LOCALAPPDATA%\GmEcuSimulator\logs\death by dps\GmEcuSimulator-prime-<timestamp>.txt - the simulator's own diary of what the archive told it to do.

Step 4 - Run DPS

1. Launch DPS. In its device-selection step, pick "GM ECU Simulator". DPS thinks it is talking to a Tactrix or MongoosePro.
2. Pick the same programming job in DPS as the archive you primed from. Step through the wizard as usual - DPS will run its full $28 / $A5 / $27 / $34 / $36 / $20 / $AE sequence against the simulator.
3. Watch the Bus log tab. You will see the entire programming flow play out frame by frame, with the calibration data captured to %LOCALAPPDATA%\GmEcuSimulator\logs\captures\.
4. If the session ends with "Programming Successfully Ended" you have a healthy round-trip. If it does not, the prime report and the bus log together should tell you exactly which step failed and what NRC the simulator returned.

Edit the prime without re-running the wizard

Once an ECU is primed you can click "Edit prime..." in the ECU Editor's settings to re-open the wizard with all of the previous choices pre-filled. The ECU is replaced atomically on Apply, so you cannot end up with half-edited state on the bus.

7. Quickstart - emulating a flash-tool target

Goal: emulate a programmable ECU for an open-source flash tool such as 6Speed.T43 (transmission control modules) or PowerPCM Flasher (engine control modules). There is no separate mode for this any more - you do it inside ECU Simulator mode by hand-configuring one ECU to match what the flash tool expects.

Step 1 - Pick ECU Simulator mode and add an ECU

1. Set the dropdown (top-right) to "ECU Simulator - J2534", then click + to add one ECU and select it. You configure it manually because a flash tool defines the wire format directly, not through a DPS-style archive - so there is no wizard here.
2. Most flash tools do not read PIDs, only download blocks, so you can ignore the PID table for this scenario and focus on the ECU Editor's settings below.

Step 2 - Configure the ECU

1. In the ECU Editor's settings, set the request CAN ID and response CAN IDs (USDT for service responses, UUDT for unsolicited messages) to whatever the flash tool expects.
2. **Flow control:** set FC.BS (Block Size) to the value the flash tool requires. 6Speed.T43 specifically requires BS=1 - it sniffs the FC tail of the simulator's reply for the literal pattern "01" to recognise a T43 TCM and will not start the kernel upload otherwise. PowerPCM Flasher is more permissive; BS=0 is fine.
3. **Security module:** pick the seed/key algorithm the target ECU would use. The dropdown lists every algorithm the simulator knows about - gm-t43-2byte for the T43, gm-e38-2byte / gm-e67-2byte for PowerPCM-compatible engine modules, gm-bypass-2byte for an unconditional bypass.

Step 3 - Run the flash tool

1. Launch the flash tool. Pick "GM ECU Simulator" as the J2534 device.
2. Run the download as you normally would. The simulator handles the full $34 (RequestDownload) / $36 (TransferData) / $37 (RequestTransferExit) sequence and

writes a capture file to %LOCALAPPDATA%\GmEcuSimulator\logs\captures\ named after the ECU, the timestamp, the base $36 address, and the byte count.

> **This is how you reverse-engineer a flash tool**
>
> The simulator is permissive - it accepts download flows that a real ECU might reject - so it is well-suited for running an unknown flash tool against it just to learn what the tool does. Turn on file logging with Append description tag and Include J2534 calls, run one session end-to-end, and the resulting CSV is essentially a transcript of the tool's expected wire flow.

8. Logging and captures

Two independent recording mechanisms run side-by-side. They serve different purposes and you can have either or both running at the same time.

The Bus log tab (live, in-memory)

The Bus log tab in the main window shows a rolling buffer of recent frames in two text panes. It is useful for watching things happen as they happen but it caps at a few thousand lines and is not persisted.

- **Log traffic checkbox (toolbar):** master gate. Off by default - in fast streaming scenarios (a $AA Fast 25Hz DPID stream, for example) the textbox would flood within seconds.
- **Hide $3E checkbox (toolbar):** suppress TesterPresent keepalives ($3E requests and $7E positive responses) from this window only. File log is unaffected.
- **Maximize checkbox (toolbar):** collapse the live-tile dashboard card and let the workspace tabs fill the full window.

The file log (persistent, on disk)

Open the **Log menu**. The first item, **Log to file**, is the master gate. When on, a dedicated background thread streams every frame as CSV to %LOCALAPPDATA%\GmEcuSimulator\logs\bus logs\bus_<timestamp>.csv. The other four items in the menu fine-tune what is captured:

- **Include J2534 calls:** write every pipe-level call (Open, Connect, StartMsgFilter, ReadMsgs, etc.) into the log.
- **Include bus traffic:** write every CAN frame in both directions.

- **Append description tag:** annotate each bus-frame line with a short human-readable tag ("SecurityAccess - RequestSeed", "ISO-TP FF (3110B) - TransferData @ 003FAFE0").
- **Collapse bulk transfers:** during long ISO-TP downloads, show the first three and last three consecutive frames and replace the middle with a single "bulk transfer collapsed: N frames hidden" marker.

Open the log folder: Log > Open folder... opens %LOCALAPPDATA%\GmEcuSimulator\logs\bus logs\ in Explorer.

Programming captures

Whenever the simulator sees a programming download ($34 / $36 / $37 sequence) it writes a .bin file containing the actual downloaded bytes. These are not opt-in - they are always written - but they only exist for sessions where a download actually happened. View them from the Captures tab (visible in DPS Simulator mode), or by opening the capture directory directly.

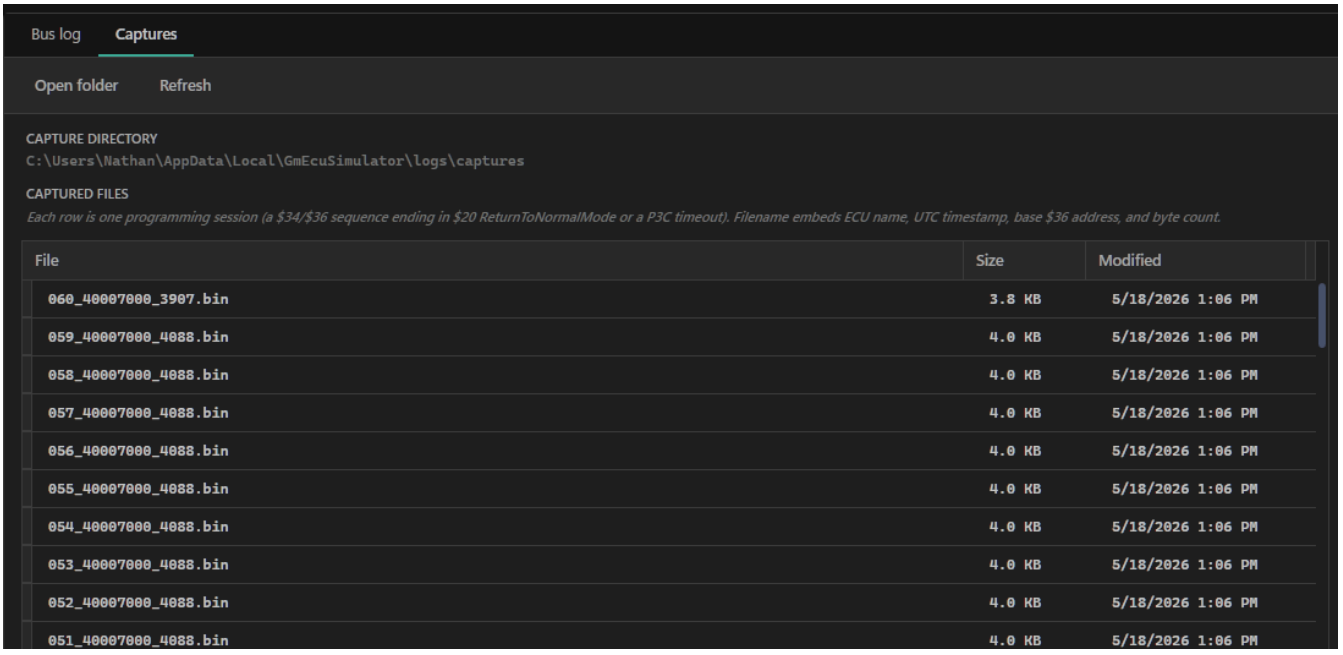

Figure: The Captures tab with one or more recorded sessions.

9. Themes

Open the **Theme menu** to pick from the bundled colour palettes. Drop your own .xaml palette file into %APPDATA%\GmEcuSimulator\Palettes\ and it appears in the same submenu after the next app restart. The theme system is shared with the sibling GM Theme Manager project - any palette built for that project loads here without modification.

10. Troubleshooting

My J2534 host does not see GM ECU Simulator in its dropdown

This is almost always one of three things:

1. **Registration is not in place.** Check the J2534 status pill in the top-right - it should read "Registered (32-bit + 64-bit)". If it reads "Not registered" or "32-bit only", run J2534 > Register as J2534 device... again.
2. **Bitness mismatch on a strict host.** If your host is 32-bit and the pill says "64-bit only" (or vice versa), the matching shim DLL did not register. The most common cause is that the install folder is missing one of the two PassThruShim DLLs. Reinstall the release zip in full.
3. **The host enumerates with the v05.00 API.** A handful of newer J2534 hosts only look at the v05.00 device list. The simulator is v04.04 only by design. Most hosts fall back to the v04.04 path when v05.00 returns empty, but a strict one will not. Workaround: pick the device by calling GetDevice("") (the v04.04 default-open path) instead of enumerating.

"GM ECU Simulator" appears in the host but the connection fails

Open the J2534 menu and click **Reset IPC pipe**. This stops and restarts the named-pipe server inside the simulator. A previously-connected host that disconnected messily can leave the pipe in a state where new opens fail; restarting the server clears that. No UAC.

The live PID values look frozen

The live PID tiles (and the ECU Editor's Live column) refresh about ten times a second from the same signal/waveform engine the bus uses, so a frozen value means that engine itself is stuck. Try **ECU > Reset ECU State** - this power-cycles every ECU, clearing programming sessions, security unlocks, download buffers, and DPID schedules.

A DPS session fails partway through

Three things to check in order:

1. **Read the prime report.** Every prime writes %LOCALAPPDATA%\GmEcuSimulator\logs\death by dps\GmEcuSimulator-prime-<timestamp>.txt - a step-by-step record of what the wizard inferred from the archive. If the report flags a step it could not infer cleanly, that is almost certainly the failure point.
2. **Read the bus log.** Turn on Log > Log to file with Append description tag before re-running. The last few lines of the CSV before the failure tell you which request the

simulator could not satisfy and which NRC it returned.

3. **Re-prime with edits.** Click Edit prime... in the ECU Editor's settings, walk through the wizard, and adjust whatever the report flagged. Apply replaces the ECU atomically - no need to clear state first.

The app reports "cancelled" after I clicked Register

That message means you clicked No on the UAC prompt. Click Register again and click Yes when the UAC dialog appears.

11. Reference

The modes and connections at a glance

Dropdown entry	Mode	Connection	ECU count	State persists?
ECU Simulator - J2534	ECU Simulator	J2534 named pipe	Many	Yes
ECU Simulator - TCP	ECU Simulator	Raw-CAN TCP socket	Many	Yes
DPS Simulator	DPS Simulator	J2534 named pipe	One (primed)	No (per-run)

Per-mode state is saved to a ".mode.json" file in the app's local data folder (ecu_simulator.mode.json for ECU Simulator, dps_simulator.mode.json for DPS Simulator) and auto-restored on the next launch into that mode. File > Save / Open let you keep additional named configs side by side.

Keyboard shortcuts

Shortcut	Action
Ctrl+N	File > New
Ctrl+O	File > Open...
Ctrl+S	File > Save
Ctrl+Shift+S	File > Save As...
Ctrl+Shift+P	Open the ECU Editor (ECU + PID editor)

Filesystem locations

All user-writable data lives under %LOCALAPPDATA%\GmEcuSimulator\.

%LOCALAPPDATA%\GmEcuSimulator\

logs\

bus logs\ bus_<timestamp>.csv (file-log output)

captures\ download captures: <ecu>-<ts>-<addr>-<bytes>.bin

death by dps\ GmEcuSimulator-prime-<timestamp>.txt (prime reports)

%APPDATA%\GmEcuSimulator\

Palettes\ drop .xaml palette files here for the Theme menu

Per-mode config file

Each mode auto-saves its state to a ".mode.json" file in the app's local data folder (ecu_simulator.mode.json, dps_simulator.mode.json) and reloads it on the next launch into that mode. The file stores the ECU list, the PID tables (mode, value source, scalar/offset, waveform), the engine character and scenario per ECU, the security-module choice, and the

per-mode flags. Use File > Save and File > Open to keep additional named configs side by side.

Registry keys written by Register.ps1

HKLM\SOFTWARE\PassThruSupport.04.04\GmEcuSim (64-bit hosts)

HKLM\SOFTWARE\WOW6432Node\PassThruSupport.04.04\GmEcuSim (32-bit hosts)

Values written into each key:

Name = GM ECU Simulator

Vendor = ECA

FunctionLibrary = full path to PassThruShim64.dll or PassThruShim32.dll

ProtocolsSupported= (DWORD bitmask)

CAN = 1

ISO15765 = 1

12. Glossary

Term	Meaning
CAN	Controller Area Network. The wire protocol every modern vehicle uses for ECU-to-ECU communication. CAN frames carry up to 8 bytes of payload at a CAN ID; the simulator's virtual bus carries the same frames in memory.
CAN ID	An 11-bit (or 29-bit) integer that identifies a CAN frame's sender or intended recipient. GM uses $7E0-$7E7 for tester-to-ECU physical requests, $7E8-$7EF for ECU-to-tester service responses.
DID	Data Identifier. A 16-bit handle for a piece of identification or configuration data on an ECU (VIN, calibration ID, hardware part number). Read with $22, written with $3B.
DPS	Diagnostic Programming System. GM's dealership-side application for reprogramming vehicle ECUs. DPS Simulator mode emulates the target ECU well enough for DPS to complete a real programming job.
DPID	Dynamic Packet Identifier. A streaming message format defined by GMW3110 mode $AA - the ECU pushes a packed snapshot of N PIDs at a fixed cadence (Slow/Medium/Fast bands).
ECU	Electronic Control Unit. The small computer inside a vehicle module (engine, transmission, body controller). One physical box, one ECU. The simulator emulates any number of ECUs at once.

Term	Meaning
Engine character	The model-specific personality that turns the engine model's operating point into derived sensor values (MAP, airflow, spark, fuelling). The simulator ships Naturally Aspirated V8 and Boosted V8; pick one per ECU in the ECU Editor.
Flash tool	Any application that reprograms an ECU. Includes GM's own DPS but also open-source projects like 6Speed.T43 (transmissions) and PowerPCM Flasher (engine modules). You emulate a target for these in ECU Simulator mode (Section 7).
GMW3110	GM's diagnostic protocol specification ("GMW3110-2010" is the 2010 revision). Defines the modes $22, $2C, $2D, $AA, $27, $34, $36, $37, $3E, etc., that the simulator implements.
ISO-TP	ISO 15765-2, the transport layer that segments diagnostic messages larger than 7 bytes across multiple CAN frames. The simulator implements full reassembly with flow control.
J2534	SAE J2534, the Windows-side standard for vehicle diagnostic interfaces. Every J2534 device exposes the same 14 functions (PassThruOpen, PassThruWriteMsgs, etc.) so any J2534-aware host can use any J2534 device interchangeably.
NRC	Negative Response Code. A one-byte error code an ECU sends when it cannot satisfy a request. Common ones: $11 serviceNotSupported, $22 conditionsNotCorrect, $33 securityAccessDenied, $78 busyResponsePending.
PassThru	The function prefix used by every J2534 entry point (PassThruOpen, PassThruWriteMsgs, etc.). "PassThru" and "J2534" are used almost interchangeably in this manual.

Term	Meaning
PID	Parameter Identifier. A 16-bit handle for a sensor value or other live datum (Engine RPM, Coolant Temp). Read with $22 (or $01 for the OBD-II subset). In ECU Simulator mode you define your own PIDs and choose each one's value source: a named engine-model signal, the row's own waveform, or none.
Prime / Prime-From-Archive	The Prime Wizard's job is to take a DPS programming archive and configure a virtual ECU that will satisfy whatever programming flow that archive expects. "Primed" means an ECU has been built from an archive (as opposed to from scratch).
Scenario	The operating point the engine model holds: Key-On Engine-Off, Idle, Cruise, or an Accel-Decel Sweep. Switching scenario ramps every signal-backed PID toward the new condition together.
SecurityAccess ($27)	The mode an ECU uses to challenge a tester before allowing programming. Tester asks for a seed, ECU returns a random number, tester computes a key from the seed, ECU validates the key. Different ECU families use different seed-to-key algorithms; the simulator ships several.
Signal	A named quantity the engine model produces (Engine RPM, MAP, Coolant Temp, ...). A PID row can draw its wire value from a signal so it tracks the live scenario instead of a fixed waveform.
SPS	Service Programming System. Older name for DPS; the bytecode inside a DPS archive is still officially called the SPS Interpreter bytecode.
Shim	The small native DLL (PassThruShim32.dll / PassThruShim64.dll) that Windows registers as the J2534 device. On the J2534 connection the

Term	Meaning
	shim forwards every PassThru call over a named pipe to the running .NET simulator process.
TCP connection	The alternative to J2534: a localhost TCP socket carrying raw CAN frames, so a non-J2534 program (e.g. a gauge simulator) can join the virtual bus directly. Selected as "ECU Simulator - TCP" in the mode dropdown.